

Bài 2- Thực hành với MDP

🎓 LAB 02 – Value Iteration and Policy Iteration in Discrete MDPs

****Environments:****

- CliffWalking-v0
- Taxi-v3

****Libraries:**** Gymnasium, NumPy

🎯 Learning Goals

By completing this lab, students will:

1. Understand ****how Bellman equations**** form the foundation of planning in Markov Decision Processes (MDPs).
2. Implement ****Value Iteration**** and ****Policy Iteration**** from first principles.
3. Observe convergence behavior and analyze how environment structure affects optimal policy learning.
4. Learn to reason about algorithm efficiency rather than simply coding it.

🧩 PART 1 – Value Iteration in CliffWalking-v0

1. Introduction

The ****CliffWalking-v0**** environment models a simple gridworld:

- The agent starts on the top-left corner.
- The goal is on the bottom-right corner.
- Any step into the "cliff" region results in a large negative reward and resets t

he episode.

Your goal:

👉 Implement **Value Iteration** to compute an optimal policy that avoids the cliff.

2. Setup

```
```python
import gymnasium as gym
import numpy as np

env = gym.make("CliffWalking-v0")
n_states = env.observation_space.n
n_actions = env.action_space.n
print("States:", n_states, "Actions:", n_actions)
```
```

3. Task Instructions

Implement **Value Iteration** **without copying code from any online source**.

🧠 THINK FIRST

Before coding, write answers (in comments) to the following:

1. What is the Bellman **optimality** equation for this problem?
2. What does the term $(\max_a Q(s,a))$ represent intuitively?
3. Why is there a trade-off between γ (discount factor) and convergence speed?

🌱 IMPLEMENTATION

```
```python
def value_iteration(env, gamma=0.9, theta=1e-6):
 """
 Compute the optimal value function and policy using Value Iteration.
 """
```

You must:

- Derive the update rule for  $V(s)$
- Apply it until convergence ( $\delta < \theta$ )
- Extract the optimal policy from  $V(s)$

"""

```
n_states = env.observation_space.n
```

```
n_actions = env.action_space.n
```

```
V = np.zeros(n_states)
```

```
TODO 1: Write the main update loop for $V(s)$
```

```
Hint: For each state, compute expected returns over all actions.
```

```
DO NOT copy any loop structure from online sources.
```

```
Instead, reason about what P , R , and γ represent here.
```

```
TODO 2: After convergence, extract the greedy policy
```

```
by choosing the action that maximizes expected future return.
```

```
TODO 3: Explain (in a comment) how your stopping criterion works.
```

```
return V, policy
```

```
...
```

#### ### 4. Evaluation

1. Print the final average state value.

2. Display the policy using arrow symbols ( $\uparrow$ ,  $\downarrow$ ,  $\leftarrow$ ,  $\rightarrow$ ).

3. Discuss (in text, not code):

- What pattern do you observe near the cliff area?
- How sensitive is the policy to different  $\gamma$  values (0.9 vs 0.99)?
- What happens if the reward for falling changes from  $-100$  to  $-10$ ?

```

```

## ## 🌱 PART 2 – Policy Iteration in Taxi-v3

### ### 1. Introduction

In **Taxi-v3**, the agent drives in a 5×5 grid:

- It must pick up a passenger at one of four fixed locations.
- Then drop them off at another.
- Each action costs -1 reward; wrong pickup/dropoff gives -10.

Your task:

👉 Implement **Policy Iteration** to find an optimal driving policy.

### ### 2. Setup

```
```python
import gymnasium as gym
env = gym.make("Taxi-v3")
n_states = env.observation_space.n
n_actions = env.action_space.n
print("Taxi states:", n_states, "Actions:", n_actions)
```
```

### ### 3. Task Instructions

This time, you will not be given any pseudocode.

#### #### 🧠 THINK BEFORE CODING

Answer (in comments) before writing your function:

1. Why do we separate **Policy Evaluation** and **Policy Improvement** steps?
2. Why must the policy evaluation step be *iterative* even for a fixed policy?
3. Under what conditions will the policy stop changing?

#### #### 🧩 IMPLEMENTATION

```
```python
def policy_iteration(env, gamma=0.9, theta=1e-6):
    """
    Implement Policy Iteration for the Taxi-v3 environment.
    """
```

Steps to reason through (write your own logic):

1. Initialize a random policy.
2. Evaluate it by repeatedly updating $V(s)$.
3. Improve it greedily.
4. Repeat until stable.

```
"""
```

```
n_states = env.observation_space.n
```

```
n_actions = env.action_space.n
```

```
policy = np.random.choice(n_actions, n_states)
```

```
V = np.zeros(n_states)
```

```
stable = False
```

```
# TODO 1: Implement Policy Evaluation
```

```
# Think about how to compute expected returns
```

```
# without using any external code reference.
```

```
# TODO 2: Implement Policy Improvement
```

```
# If the best action changes, mark policy as unstable.
```

```
# TODO 3: Add inline reasoning (in comments)
```

```
# for why this loop converges.
```

```
return V, policy
```

```
...
```

4. Testing and Analysis

Run both algorithms and compare:

```
```python
```

```
V_vi, policy_vi = value_iteration(env)
```

```
V_pi, policy_pi = policy_iteration(env)
```

```
print("Mean Value (VI):", np.mean(V_vi))
```

```
print("Mean Value (PI):", np.mean(V_pi))
```

```
print("Different actions:", np.sum(policy_vi != policy_pi))
```

...

#### #### 🧠 REFLECT

- Which algorithm converged faster? Why?
- How do environment size and stochasticity affect convergence?
- Which step (evaluation vs improvement) dominates computational cost?

---

#### ## 💬 Submission Requirements

Each student must submit **one Jupyter Notebook (.ipynb)** that includes:

1. Completed implementations for both algorithms (with reasoning comments).
2. Printed results for both environments.
3. Analytical discussion — *not* copied text — on:
  - How rewards and discount factors shape behavior.
  - Why Value Iteration and Policy Iteration both reach the same optimal policy.
4. Answers to all reasoning questions above.

---

#### ## 📊 Grading (10 points)

Criterion	Description	Points
Correct Implementation	Both algorithms run correctly	5
Analytical Reasoning	Clear written logic & comments	2
Discussion & Insights	Deep explanation of outcomes	2
Clarity & Presentation	Code structure and readability	1

---

#### ## ⚠️ Academic Integrity

This lab is designed so that **\*\*copying code or using AI assistants (ChatGPT, Copilot, etc.) will not help you\*\***:

- You must **\*\*explain each line\*\*** and **\*\*justify your algorithmic choice\*\***.
- Submissions without reasoning comments will not receive full credit.
- The instructor may ask you to explain your logic verbally.

---

## ## 📖 References

- Sutton & Barto, *\*Reinforcement Learning: An Introduction\**, Chapter 4.
- Gymnasium Documentation – Environments: [<https://gymnasium.farama.org/environments/>](<https://gymnasium.farama.org/environments/>)
- Pieter Abbeel, *\*Markov Decision Processes and Exact Solution Methods\** (U C Berkeley).

---

> 🧑‍🏫 **\*Instructor tip:\***

> For grading, remove points if the student's notebook only contains working code but lacks any reasoning or explanation. Reward thought, not syntax.